MINISTRY OF EDUCATION AND RESEARCH

Technical University of Moldova

Faculty of Computers, Informatics, and Microelectronics

Department of Software Engineering and Automation

PLATFORMĂ MACOS PENTRU PROTOTIPAREA VIZUALĂ ÎN TIMP REAL ȘI GENERAREA INTERFEȚELOR PENTRU APLICAȚII IOS

MACOS PLATFORM FOR REAL-TIME VISUAL PROTOTYPING AND INTERFACE GENERATION FOR IOS APPLICATIONS

Undergraduate Internship Report

Student: _____ Dragomir Mîndrescu, gr. FAF-221

Coordinator: _____ Gaidău Mihai, univ. assist.

Chisinau, 2026

Adnotare

Lucrarea intitulată **Platformă macOS pentru prototiparea vizuală în timp real și generarea interfețelor pentru aplicații iOS** examinează proiectarea și implementarea unei platforme native *macOS* pentru prototiparea vizuală a aplicațiilor *iOS*. Prototipul implementat, **SwiftBuilder**, combină compoziția multi-ecran, o bibliotecă de componente organizată pe categorii, editarea contextuală a proprietăților, previzualizarea dependentă de dispozitiv, configurarea navigației, persistența în format JSON, exportul codului SwiftUI și execuția nativă prin intermediul unei aplicații de rulare pentru *iOS*. Această combinație răspunde decalajului existent dintre machetele statice de interfață, constructorii no-code generici și efortul manual necesar lucrului direct în ecosistemul Apple.

Raportul este structurat în conformitate cu cerințele instituționale pentru acest tip de lucrare. Introducerea definește relevanța temei, problema abordată, obiectivul general și metodologia utilizată. Capitolul 1 prezintă analiza domeniului prin examinarea stadiului actual al dezvoltării low-code și al prototipării native pentru *iOS*, a nevoilor actorilor implicați, a soluțiilor concurente, a poziționării SWOT și a obiectivelor proiectului. Capitolul 2 formulează specificația cerințelor prin identificarea actorilor, delimitarea domeniului de aplicare și definirea cerințelor funcționale și nefuncționale. Capitolul 3 prezintă modelarea sistemului prin diagrame UML comportamentale și structurale, precum și prin modelele de persistență și distribuire. Capitolul 4 descrie tehnologiile utilizate pentru implementare, arhitectura cu două ținte, stratul comun de randare, mecanismul de export al codului și fluxul de validare în simulator.

Contribuția principală a lucrării constă în documentarea unei arhitecturi coerente pentru prototiparea vizuală nativă a interfețelor pe platformele Apple. Raportul demonstrează că baza curentă de cod susține deja un set consistent de funcționalități, inclusiv proiecte multi-ecran, componente reutilizabile, șabloane de ecran, anulare și refacere a acțiunilor, comenzi rapide de la tastatură, persistență JSON compatibilă retroactiv și randare partajată între aplicația de construire și aplicația de rulare. În acest mod, proiectul depășește nivelul unui simplu editor de machete și se apropie de un mediu credibil de autorat pentru proiectarea timpurie a aplicațiilor *iOS*.

Cuvinte-cheie: SwiftBuilder, constructor vizual pentru aplicații *iOS*, export SwiftUI, validare prin aplicație de rulare, *macOS*

Abstract

The thesis titled **macOS Platform for Real-Time Visual Prototyping and Interface Generation for iOS Applications** examines the design and implementation of a native macOS platform for visual prototyping of iOS applications. The implemented prototype, **SwiftBuilder**, combines multi-screen composition, a categorized component library, contextual property editing, device-aware preview, navigation wiring, JSON persistence, SwiftUI code export, and native execution through a companion iOS runner application. This combination addresses the gap between static interface mockups, generic no-code builders, and the full manual effort required by the native Apple development toolchain.

The report is organized according to the official structure required for this document type. Chapter 1 presents the domain analysis by examining the current state of no-code development, native iOS prototyping, stakeholder needs, competing solutions, the SWOT position of the proposed system, and the project objectives. Chapter 2 formalizes the requirements specification through actor identification, scope definition, and functional and non-functional requirements. Chapter 3 presents the system modeling artifacts through UML-based behavioral, structural, deployment, and persistence views. Chapter 4 focuses on system implementation by describing the technologies used, the dual-target architecture, the shared rendering layer, the code-export pipeline, and the simulator-backed validation workflow.

The main contribution of the work is the documentation of a coherent architecture for native visual interface prototyping on Apple platforms. The report shows that the current codebase already supports a substantial feature set, including multi-screen projects, reusable components, screen templates, undo/redo, keyboard shortcuts, backward-compatible JSON persistence, and shared rendering between the macOS builder and the iOS runner. In this way, the thesis moves beyond a simple mockup editor and toward a credible native authoring environment for early iOS application design.

Keywords: SwiftBuilder, visual iOS app builder, SwiftUI code export, runner-based validation, macOS

TABLE OF CONTENTS

LIST OF FIGURES . . . . . **v**

LIST OF TABLES . . . . . **vi**

INTRODUCTION **1**

1 DOMAIN ANALYSIS **3**

1.1 Current State of the Field in 2026 3

1.2 Problem and Stakeholder Impact 3

1.3 Competitive Analysis 4

1.4 SWOT Matrix 5

1.5 Target Actors 6

1.6 Solution Proposal 7

1.7 Architectural Rationale of the Proposed Direction 7

1.8 SMART Objectives 8

1.9 Expected Academic and Practical Contribution 9

2 REQUIREMENTS SPECIFICATION **10**

2.1 Actor Model and System Scope 10

2.2 Functional Requirements 11

2.3 Non-Functional Requirements 13

2.4 Constraints and Assumptions 14

2.5 Requirements Traceability 15

2.6 Acceptance Criteria for Core Workflows 15

3 SYSTEM MODELING **17**

3.1 Modeling Strategy 17

3.2 Use Case Model 17

3.3 Activity Model of the Main Workflow 18

3.4 Sequence Model of Runner Execution 19

3.5 Class Model of the Core Types 19

3.6 Deployment Model 20

3.7 Persistence Model 21

3.8 High-Level Architecture Model 22

3.9 Consistency Between Models and Requirements 23

3.10 Modeling Synthesis 24

4 SYSTEM IMPLEMENTATION **25**
4.1 Implementation Context 25
4.2 Technology Stack Overview 25
4.3 Justification of the Native Apple Stack 26
4.4 Implementation Architecture 26
4.5 Technical Constraints of the Current Implementation 27
4.6 Technology Selection Criteria 27
4.7 Role of the Stack in the End-to-End Workflow 28
CONCLUSIONS **29**
BIBLIOGRAPHY **31**

LIST OF FIGURES

3.1 Use Case Diagram of the Main Authoring and Output Workflow 18
3.2 Activity View of the Main Builder Workflow 18
3.3 Sequence Diagram of the Runner Launch Flow 19
3.4 Class Diagram of the Core Builder, Export, and Runner Types 20
3.5 Deployment Diagram of the Local Builder and Runner Environment 21
3.6 Persistence Model of the Builder Project JSON 21
3.7 High-Level Architecture of SwiftBuilder 22

LIST OF TABLES

1.1 Competitive Analysis of Relevant Tools 5
1.2 SWOT Matrix for SwiftBuilder 6
1.3 SMART Objectives of the Project 8

2.1 Actor Model of SwiftBuilder 10
2.2 Traceability from User Needs to Requirements, Models, and Evidence 15
2.3 Acceptance Criteria for the Core Workflows 16

4.1 Technology Stack and Justification 25

INTRODUCTION

By 2026, low-code and no-code development have become established topics in software engineering practice and research. Organizations increasingly expect visual environments to shorten iteration loops, broaden participation in product definition, and reduce the translation loss between requirements, design artifacts, and implementation decisions [1], [2], [3], [4]. At the same time, native *iOS* development still depends on a comparatively technical toolchain built around Swift, SwiftUI, Xcode, and Apple-specific platform conventions [5], [6], [7], [8]. This combination creates a persistent tension: interface ideas can be described quickly, but producing realistic native prototypes still often requires manual coding and full toolchain knowledge.

The present report addresses this gap through **SwiftBuilder**, developed under the subject **macOS Platform for Real-Time Visual Prototyping and Interface Generation for iOS Applications**. The implemented system is a dual-target Xcode project composed of a native *macOS* builder and a companion *iOS* runner application. Together, these targets support multi-screen visual authoring, contextual property editing, realistic device preview, screen templates, navigation wiring, JSON-based persistence, SwiftUI code export, and simulator-backed validation through shared rendering logic.

Problem Statement. The core problem addressed by the thesis is the lack of a compact workflow that connects visual authoring, native preview, and structured handoff for *iOS* interfaces. Design-oriented tools are effective for layout exploration and communication, but they usually stop at static or semi-interactive mockups. Generic no-code builders often optimize for platform-agnostic delivery rather than for native Apple-platform fidelity. Xcode and SwiftUI provide complete native control, but they require programming effort even when the immediate goal is only to explore screen structure, navigation, and interface behavior [9], [10]. This gap matters because early interface validation is not only a design concern, but also a requirements and architecture concern. Teams and independent creators need to test navigation, content hierarchy, component combinations, and device-scale behavior before investing heavily in full application logic.

Relevance and Objective. The project is motivated by the need for a local-first, native *macOS* environment that shortens the path from concept to realistic *iOS* prototype while remaining close to the Apple toolchain. The objective is not to replace end-to-end software development, but to provide a focused authoring environment in which interface structure, screen-to-screen navigation, visual styling, code export, and native execution can be explored within one coherent system. The current implementation already supports multi-screen project composition, a component library containing 21 reusable interface elements grouped into 6 categories, contextual property editing, preview on 7 Apple device presets, JSON

persistence, backward-compatible runner loading, SwiftUI code export, and runtime validation through a companion runner application and automated simulator launch.

Methodology. To establish the analytical and technical basis of the thesis, the work combines several complementary methods: review of academic and technical literature on low-code development, prototyping, and native *iOS* tooling; comparative analysis of existing design, no-code, and native development tools; direct inspection of the current SwiftBuilder implementation; formalization of functional and non-functional requirements from the implemented feature set; and system modeling through UML and architecture-oriented views of the core workflows, data structures, and deployment environment. This methodology keeps the report grounded in the real system while also framing the topic in relation to broader software-engineering concerns such as native fidelity, prototyping efficiency, maintainability, and model consistency.

Report Structure. The report is organized into four main chapters after the introduction. Chapter 1 presents the domain analysis by examining the current state of no-code and native *iOS* prototyping, the problem and stakeholder impact, competing solutions, the SWOT position of the proposed system, and the objectives of the project. Chapter 2 presents the requirements specification through actor identification, scope definition, and functional and non-functional requirements. Chapter 3 presents the system modeling artifacts through UML behavior views, structural views, deployment interpretation, and the persistence model. Chapter 4 presents the implementation view by justifying the selected technologies and documenting the actual architecture of SwiftBuilder. The report ends with conclusions that summarize the current contribution and outline the most relevant directions for later extension.

1 DOMAIN ANALYSIS

1.1 Current State of the Field in 2026

The thesis topic lies at the intersection of low-code development, interface prototyping, and native Apple-platform tooling. This field remains relevant because contemporary software development values shorter validation cycles, stronger collaboration between design and engineering, and more reliable transitions from concept to implementation. Research on low-code platforms shows that visual development environments are no longer treated only as simplified educational tools; they are now discussed as serious software-engineering instruments related to productivity, model-driven development, and citizen development [1], [2], [3], [4], [11].

At the same time, the literature also shows that low-code systems vary substantially in maturity, scope, and technical depth. Some prioritize interface composition, some emphasize data and backend modeling, and others optimize for code generation or rapid deployment. This observation is important for the present thesis because it discourages generic positioning. The value of the proposed system must be defined in relation to a precise gap rather than through broad marketing language about no-code productivity [1], [2].

For native iOS prototyping, that gap remains visible in 2026. SwiftUI has reduced some of the boilerplate of interface development, but native prototyping still presupposes knowledge of programming, Xcode configuration, navigation structure, and Apple interface conventions [5], [6], [7]. In parallel, product teams and independent developers increasingly need to validate interface ideas earlier, when requirements are still fluid and rework costs are still avoidable. Prototyping remains valuable precisely because it supports communication, requirement refinement, and architectural clarification before the final implementation is stabilized [9], [10], [12].

1.2 Problem and Stakeholder Impact

The central problem addressed by the thesis is the lack of a compact native workflow that connects visual editing, multi-screen interface modeling, and executable iOS validation. Design tools such as Figma and Sketch are effective for static composition and review, but they do not provide a direct native runtime context as part of the same authoring loop [13], [14]. On the other hand, Xcode and SwiftUI support full native execution, but they assume coding effort even when the immediate goal is to explore screen structure, component combinations, navigation, or visual presentation [6], [7].

This problem affects several stakeholder groups:

– **Designers** need to validate interfaces beyond static layouts, especially when device proportions,

screen transitions, and runtime behavior matter.

- **Independent developers** need to test product ideas quickly without manually implementing every interface experiment in SwiftUI from the beginning.
- **Students and educators** benefit from a visual bridge between interface theory and native implementation concepts.
- **Product stakeholders** need artifacts that are more realistic than slide decks and static boards, but lighter than full application development.

The impact of the problem is not limited to convenience. Weak early validation increases rework, slows feedback cycles, and preserves a divide between design artifacts and implementation artifacts. In a native mobile context, the inability to test screen flows, navigation targets, component behavior, and device-specific presentation early enough can lead to both design and engineering inefficiency.

1.3 Competitive Analysis

Following distinctions discussed in low-code and model-driven-development literature [1], [2], [11], the competitive analysis compares five concrete tools that are relevant to the thesis context. Vendor documentation is used in this section only to confirm current product positioning, execution model, and target audience [7], [13], [15], [16], [17]. The purpose of the comparison is to identify the feature combination that remains weakly served: multi-screen visual authoring with native iOS orientation, code export, and direct runtime validation.

Table 1.1 - Competitive Analysis of Relevant Tools

Tool	Primary Orientation	Low-Code Entry	Native iOS Focus	Executable Output	Main Limitation for This Thesis Context
Figma	Interface design and mockups	High	No	Prototype-only	Strong for visual exploration and team communication, but weak in native execution, structured navigation, and code-oriented handoff.
FlutterFlow	Cross-platform no-code builder	High	Partial	Generated project	Focuses on broader application generation rather than on a lightweight SwiftUI-centered builder aligned with Apple-native preview behavior.
Adalo	Generic no-code application building	High	No	Platform-specific deployment	Prioritizes platform-agnostic application assembly, which reduces native Apple-platform fidelity and limits close alignment with SwiftUI workflows.
Judo	SwiftUI-oriented commercial tooling	Medium	Partial	Native-oriented output	Closer to the thesis domain, but commercial, opinionated, and less suitable as a transparent local-first academic prototype.
Xcode	Native development environment	Low	Yes	Full native app	Offers complete native control, but the entry barrier remains high for rapid interface experimentation and design-driven iteration.

The analysis highlights a useful niche. Design-first tools are strong in layout exploration, but weak in native execution and structured export. Xcode is strong in native execution, but weak as a low-friction visual environment. Generic no-code tools reduce the entry barrier, but usually move away from native Apple-platform behavior. The proposed system is intended to occupy the space between these categories by emphasizing native SwiftUI-oriented authoring, structured persistence, code export, and runner-based validation.

1.4 SWOT Matrix

The SWOT analysis summarizes the strategic position of the proposed system in its present form.

Table 1.2 - SWOT Matrix for SwiftBuilder

Strengths	Weaknesses
– Native macOS implementation aligned with the Apple ecosystem. – Multi-screen authoring with navigation wiring and device-aware preview. – Dual output paths through SwiftUI code export and runner-based execution. – Shared rendering layer between builder and runner, which improves visual consistency. – Zero external dependencies and local ownership of project files.	– The system remains limited to a curated component vocabulary rather than arbitrary custom UI. – Code export focuses on interface structure and navigation, not on business logic or backend integration. – The workflow is bounded to macOS, Xcode, and the Apple simulator ecosystem. – Collaborative editing, cloud synchronization, and shared project management are not part of the current design.
Opportunities	**Threats**
– Educational use as a bridge into SwiftUI and native interface architecture. – Extension toward richer components, data binding, and stronger code-generation capabilities. – Increased demand for tools that connect design and engineering more directly. – Potential use in rapid product validation and startup-style prototyping workflows.	– Rapid SwiftUI evolution may require continuous maintenance of generated code and preview rendering. – Commercial platforms may absorb adjacent use cases with larger ecosystems and marketing reach. – Feature growth may reduce clarity if the component model and architecture are not kept disciplined. – Apple-platform dependence limits the portability of the approach to non-Apple environments.

The SWOT result supports the thesis direction. The project is not strongest when evaluated as a universal no-code platform, but it becomes more defensible when evaluated as a native-focused interface builder that links visual authoring, structured export, and runtime validation.

1.5 Target Actors

The domain analysis identifies three primary user segments:

- independent iOS developers who need faster early-stage interface iteration and navigation testing;
- designers who collaborate with Apple-platform teams and need more realistic prototypes than static boards;
- students and educators who benefit from a visual entry point into SwiftUI-oriented interface construction.

These segments share a common need: a tool that reduces the distance between concept and native iOS prototype without requiring full implementation effort from the first step. Their expectations differ, but all of them benefit from a workflow that supports screens, components, navigation, and executable preview in one environment.

A representative usage profile is an independent iOS developer who needs to draft an onboarding flow, explore a login screen, connect it to a profile screen, preview the result on several Apple device profiles, and export SwiftUI code for later refinement. In such a scenario, the main pain points are repetitive layout

coding, slow experimentation in Xcode, and the absence of a lightweight local tool that remains close to native iOS behavior.

1.6 Solution Proposal

The proposed solution is SwiftBuilder, a macOS-native visual environment for building multi-screen iOS interface prototypes. The system does not attempt to replace the full software-development lifecycle. Instead, it focuses on a more precise and technically defensible promise: enable users to create screens visually, manage a component-based canvas, configure navigation, preview the result on realistic device frames, persist the project as structured JSON, export standalone SwiftUI code, and validate the outcome through a runner application and the iOS Simulator.

This proposal improves on existing alternatives in a targeted way:

- compared with design-first tools, it adds native-facing preview, navigation modeling, and executable output;
- compared with Xcode, it lowers the barrier for early visual experimentation and screen-level exploration;
- compared with generic no-code systems, it keeps the workflow centered on native Apple-platform presentation and SwiftUI-oriented output.

The value of the system therefore lies in bridging a precise gap between static interface design, no-code composition, structured export, and native iOS validation.

1.7 Architectural Rationale of the Proposed Direction

The proposed direction is supported by four architectural choices.

First, the system is local-first rather than cloud-first. Project data is saved as JSON and mirrored locally for the runner workflow. This keeps authorship transparent, reduces infrastructure overhead, and aligns with the fact that the primary validation target is the local Apple simulator environment.

Second, the architecture is dual-target and shared-layer based. The macOS builder and the iOS runner are separate targets, but both reuse the same model and rendering primitives through the shared layer. This design is important because it reduces the mismatch between what is edited in the builder and what is rendered in execution.

Third, the core interaction state is centralized in one coordination layer. Screen management, block selection, device settings, export actions, and undo snapshots are managed through a single shared source of truth. This simplifies interaction logic and makes the system easier to reason about than a scattered collection of partially synchronized view states.

Fourth, the system provides two complementary output paths: SwiftUI code export and runner-

based execution. Code export supports structured handoff toward later implementation, while the runner supports immediate native validation of the current project. These two paths are not redundant; together, they form the practical value of the platform.

Taken together, these design choices show that the system is not just a visual editor with decorative device frames. It is a builder architecture with persistence, transformation, and execution layers that are coherent enough to justify detailed modeling and technical discussion.

1.8 SMART Objectives

Based on the domain analysis, the project goal can be formulated as follows:

to provide a macOS-native visual environment for designing, exporting, and validating multi-screen iOS interface prototypes through a coherent builder-runner architecture.

This goal is decomposed into the SMART objectives shown in Table 1.3.

Table 1.3 - SMART Objectives of the Project

ID	Objective	SMART Interpretation
SO1	Provide a native visual authoring environment that supports screen creation, selection, duplication, renaming, and deletion inside one project workspace.	Specific to the core authoring problem, measurable through the implemented screen-management operations, achievable with the current builder architecture, relevant to rapid interface prototyping, and demonstrable in the implemented system.
SO2	Provide a reusable component library and contextual editing workflow covering at least 21 interface components grouped into 6 categories.	Specific to interface construction, measurable through the implemented component catalog and inspector controls, achievable in the current codebase, relevant to builder productivity, and directly observable in the prototype.
SO3	Support native-oriented preview and flow validation through 7 Apple device presets, 2 appearance modes, and screen-to-screen navigation targets.	Specific to runtime-oriented prototyping, measurable through the implemented preview settings and navigation behavior, achievable with the current shared rendering model, relevant to the thesis contribution, and verifiable in the archived prototype.
SO4	Provide two output paths: structured SwiftUI code export and native execution through a runner application.	Specific to the main technical differentiation of the project, measurable through the implemented export and runner subsystems, achievable in the current architecture, relevant to the design-to-validation workflow, and evidenced by the prototype behavior.
SO5	Maintain a local-first and modular architecture based on native Apple frameworks, JSON persistence, and zero external runtime dependencies.	Specific to the architectural constraints of the system, measurable through the selected technologies and module boundaries, achievable within the current implementation, relevant to maintainability and reproducibility, and verifiable in the archived codebase.

The SMART decomposition is useful beyond formal compliance. It constrains the thesis to observ-

able system capabilities and reduces the risk of vague claims about the value of the platform.

1.9 Expected Academic and Practical Contribution

The expected contribution of the thesis is twofold. From a practical perspective, the project provides a structured workflow for a part of the iOS development lifecycle that is still underserved: early interface prototyping that is more realistic than static design boards, yet lighter than full manual SwiftUI implementation from the outset. The combination of screen composition, navigation wiring, shared preview rendering, code export, and simulator-backed execution shows that this workflow can be integrated into one coherent system.

From an academic perspective, the thesis contributes by documenting the architecture of this workflow explicitly. The project is positioned against low-code research, prototyping literature, and native iOS constraints, while its state management, shared rendering, persistence, export, and execution paths are described in a way that can be critiqued or extended in later work. This is particularly valuable in a domain where tools are often described through product language rather than through explicit system models and technical boundaries.

The project also has educational value. Because it is built around SwiftUI-oriented components and Apple device presets, it can serve as a bridge for students who understand interface goals conceptually but are not yet comfortable with the full native toolchain. That educational perspective does not replace the product rationale, but it strengthens the thesis relevance in the university context.

The domain analysis therefore confirms three conclusions. First, the topic remains relevant because the demand for faster and more realistic early validation continues to grow. Second, the proposed system is differentiated enough when evaluated against its precise niche. Third, the current codebase now provides a sufficiently strong architectural basis for a report that aims to be both technically honest and academically solid.

2 REQUIREMENTS SPECIFICATION

This chapter translates the results of the domain analysis into a formal specification of what the system must do and under which quality constraints it must operate. The specification is derived from the implemented architecture of SwiftBuilder rather than from an abstract future platform. For that reason, the chapter formalizes the actor model, the system boundaries, the functional requirements, the non-functional requirements, and the main traceability links between user needs and technical responses.

2.1 Actor Model and System Scope

The system remains intentionally simple from the actor perspective. It is a single-user authoring environment rather than a collaborative cloud platform. The actor model is nonetheless explicit because the report must identify who interacts with the system, which external tools participate in the workflow, and where the system boundary ends.

Table 2.1 - Actor Model of SwiftBuilder

Actor / System	Category	Responsibility in the Implemented Workflow
User	Primary human actor	Creates screens, edits components, configures navigation, exports code, and validates the prototype.
Local File System	Supporting external system	Stores saved JSON projects, exported Swift files, and the mirrored project copy used by the runner flow.
Xcode build tool	Supporting external tool	Builds the runner application target for simulator execution.
`simctl` / iOS Simulator	Supporting external toolchain	Discovers devices, boots the simulator, installs the runner application, and launches the prototype.
Runner application	Supporting runtime application	Loads exported projects and renders the prototype as a navigable iOS application.

The implemented scope includes the following validated capabilities:

- multi-screen visual composition with screen creation, selection, renaming, duplication, and deletion;
- a categorized component library with search, outline view, and drag-and-drop reordering;
- contextual property editing for content, typography, colors, layout, effects, and navigation;
- realistic device preview for 7 Apple device presets, 2 appearance modes, and zoom control;
- local JSON project persistence with backward-compatible runner loading;
- SwiftUI code export for both single-screen and multi-screen projects;
- native execution through the runner application and automated simulator launch;
- undo/redo, keyboard shortcuts, and template-driven screen creation.

The following capabilities remain outside the current system boundary:

- backend logic generation and data-model generation;

– cloud collaboration, shared workspaces, and online synchronization;
– arbitrary custom component programming inside the builder itself;
– end-to-end production application generation including business logic and network integration.

These boundaries are important because they keep the report technically honest. The system is already substantial, but it is still a native visual builder focused on interface authoring, navigation modeling, export, and preview rather than a general-purpose full-stack no-code platform.

2.2 Functional Requirements

The functional requirements are grouped by the major responsibilities of the implemented system.

FR 1. Project and Screen Management

– **FR 1.1.** The system shall create a new project initialized with a starter screen.
– **FR 1.2.** The system shall allow the user to add, rename, duplicate, select, and delete screens.
– **FR 1.3.** The system shall prevent deletion of the final remaining screen.
– **FR 1.4.** The system shall allow the user to create a screen from a predefined template or as a blank starter screen.

FR 2. Component Library and Canvas Composition

– **FR 2.1.** The system shall provide 21 reusable component types organized into 6 categories.
– **FR 2.2.** The component library shall support search by component name or description.
– **FR 2.3.** The system shall allow the user to insert a component after the currently selected block or at the end of the screen if no block is selected.
– **FR 2.4.** The system shall provide an outline view of the current screen.
– **FR 2.5.** The outline view shall support block reordering through drag-and-drop and explicit move actions.

FR 3. Property Editing and Block Operations

– **FR 3.1.** The system shall provide a contextual inspector for the currently selected block.
– **FR 3.2.** The inspector shall expose content controls specific to the selected component type.
– **FR 3.3.** The inspector shall expose typography, color, layout, and style/effects controls where applicable.
– **FR 3.4.** The system shall allow duplication, reset, and deletion of the selected block.
– **FR 3.5.** The system shall preserve block identity when resetting a block to its template defaults.

FR 4. Navigation Modeling

– **FR 4.1.** The system shall allow a block to reference an optional navigation target screen.
– **FR 4.2.** The navigation target selector shall be available only when more than one screen exists in the project.

- **FR 4.3.** The outline view shall visually indicate blocks that have navigation targets.
- **FR 4.4.** The runner application shall navigate to the target screen when a wired block is activated.

FR 5. Device Preview and Appearance Configuration

- **FR 5.1.** The system shall provide device-framed preview for the following presets: iPhone SE, iPhone 16, iPhone 16 Pro, iPhone 16 Pro Max, iPhone Air, iPad mini, and iPad Air 11".
- **FR 5.2.** The system shall provide light and dark appearance modes.
- **FR 5.3.** The system shall allow zoom control of the workspace within the implemented range.
- **FR 5.4.** The preview area shall display the selected screen name, device, appearance mode, and component count.

FR 6. Persistence and Loading

- **FR 6.1.** The system shall save the current project as a JSON document.
- **FR 6.2.** The JSON document shall store project metadata, screen definitions, component properties, and navigation targets.
- **FR 6.3.** The export workflow shall mirror the saved project to a host Documents location used by the runner application.
- **FR 6.4.** The runner shall load projects from a priority order that includes environment-provided directories, mirrored host storage, app documents, and bundle fallback.
- **FR 6.5.** The loader shall support legacy single-screen exports through backward-compatible decoding.

FR 7. SwiftUI Code Export

- **FR 7.1.** The system shall export the current project as standalone SwiftUI source code.
- **FR 7.2.** For multi-screen projects, the export shall generate a root `NavigationStack` and separate view structures for each screen.
- **FR 7.3.** For single-screen projects, the export shall generate the corresponding screen view only.
- **FR 7.4.** The export shall translate supported components and navigation targets into SwiftUI equivalents.
- **FR 7.5.** The system shall allow the user to select the output file location through the native save dialog.

FR 8. Runner Application Rendering

- **FR 8.1.** The runner application shall load the newest available project export.
- **FR 8.2.** The runner shall reconstruct screens and blocks from the JSON representation.
- **FR 8.3.** The runner shall render screens using the same shared block-rendering logic as the builder.
- **FR 8.4.** The runner shall provide loading, error, and empty states.
- **FR 8.5.** The runner shall expose a debug overlay that can be toggled on device.

FR 9. Automation and Productivity Features

- **FR 9.1.** The system shall support one-click simulator execution of the current project.
- **FR 9.2.** The simulator workflow shall include JSON save, project mirroring, runner build, installation, and application launch.
- **FR 9.3.** The system shall support undo and redo for structural editing actions through snapshot-based state restoration.
- **FR 9.4.** The system shall provide keyboard shortcuts for save, export, run, duplicate, delete, template gallery, new screen, undo, and redo actions.

Taken together, the functional requirements show that the system is not only a visual editor. It is a multi-screen authoring environment with persistence, code transformation, and native validation capabilities.

2.3 Non-Functional Requirements

The non-functional requirements define the quality expectations and engineering constraints of the implemented system.

NFR 1. Native Fidelity

- **NFR 1.1.** For supported component types, the builder preview and the runner shall use the same shared rendering definitions.

NFR 2. Performance and Responsiveness

- **NFR 2.1.** Preview refresh after a single edit operation shall complete within 1 second for ordinary projects of up to 5 screens and 100 blocks.
- **NFR 2.2.** Save and export operations shall complete within 3 seconds for ordinary projects of up to 5 screens and 100 blocks.

NFR 3. Reliability and Failure Transparency

- **NFR 3.1.** Invalid project files, simulator failures, export errors, and build failures shall produce visible feedback through an alert, status message, or debug screen rather than silent failure.

NFR 4. Backward Compatibility

- **NFR 4.1.** The persistence format shall continue to load earlier single-screen exports through compatibility logic rather than through hard failure.

NFR 5. Usability and Discoverability

- **NFR 5.1.** Primary commands for save, export, run, undo, redo, and screen creation shall be available through visible controls in the builder window.
- **NFR 5.2.** The system shall provide keyboard shortcuts for save, export, run, duplicate, delete, template gallery, new screen, undo, and redo actions.

NFR 6. Maintainability and Modularity

- **NFR 6.1.** Builder logic, shared models, shared rendering, code generation, and runner logic shall remain separated into clear modules.

NFR 7. Privacy and Local Ownership

- **NFR 7.1.** The main workflow shall remain local-first, and project files shall stay under user control without mandatory cloud services or remote storage.

NFR 8. Platform Compatibility

- **NFR 8.1.** The builder shall operate on modern macOS systems with Xcode command-line tools.
- **NFR 8.2.** The runner shall target the iOS simulator environment.

NFR 9. Dependency Minimization

- **NFR 9.1.** The application shall remain free of external runtime dependencies and shall rely on native Apple frameworks only.

NFR 10. Export Quality

- **NFR 10.1.** The generated SwiftUI code shall compile without manual structural corrections for supported builder constructs.

These non-functional requirements are especially important for the thesis subject because the value of the system depends not only on feature count, but also on fidelity, clarity, maintainability, and trustworthiness.

2.4 Constraints and Assumptions

The current system operates under several explicit constraints and assumptions:

- the workflow is intentionally centered on Apple platforms and relies on macOS, Xcode, SwiftUI, and the iOS Simulator;
- the application is local-first and single-user, not collaborative or cloud-hosted;
- the code generator produces interface-oriented SwiftUI code rather than full application logic;
- the component vocabulary is finite and curated, which improves consistency but limits arbitrary customization;
- no backend integration, persistent network model, or data-binding layer is currently generated by the builder.

These assumptions are not weaknesses of documentation; they are necessary boundaries that keep the report aligned with the real system.

2.5 Requirements Traceability

The final step of the requirements chapter is to connect user needs to the technical responses that are modeled in the next chapter and justified in the implementation chapter.

Table 2.2 - Traceability from User Needs to Requirements, Models, and Evidence

User Need	Primary Requirements	Models and Evidence in the Report
Rapid multi-screen interface exploration	FR 1, FR 2, FR 3, FR 5	Use case and activity views in Figures 3.1 and 3.2; architecture discussion in Section 4.4.
Realistic native preview and navigation validation	FR 4, FR 5, FR 8, FR 9	Sequence and deployment views in Figures 3.3 and 3.5; runtime technology rationale in Section 4.4.
Structured handoff toward implementation	FR 6, FR 7, NFR 10	Persistence and class views in Figures 3.6 and 3.4; export discussion in Section 4.3.
Low-friction authoring workflow	FR 2, FR 3, FR 9, NFR 5	Activity view in Figure 3.2; builder technology and interaction support discussed in Sections 4.2 and 4.4.
Stable architectural base for later extension	NFR 4, NFR 6, NFR 9	High-level architecture in Figure 3.7; implementation constraints and architecture notes in Sections 4.4 and 4.5.

The requirements specification therefore establishes a precise contract for the remaining chapters. The next chapter explains how these requirements are modeled formally, while the implementation chapter shows how they are realized through the current architecture of SwiftBuilder.

2.6 Acceptance Criteria for Core Workflows

The requirements become more useful when they are complemented by explicit acceptance criteria. In the context of the present thesis, acceptance criteria do not replace later testing activities, but they define what must be observable in order to claim that the implemented workflow is operational. This is important because the project combines visual authoring, persistent storage, code export, and simulator-backed execution, which means that success cannot be reduced to the presence of interface controls alone.

Table 2.3 - Acceptance Criteria for the Core Workflows

Workflow	Acceptance Criterion	Expected Observable Result
Project and screen management	The user creates a new project, adds two extra screens, renames one of them, duplicates another, and attempts to delete screens until only one remains.	The workflow succeeds for create, rename, and duplicate operations, while deletion of the final remaining screen is prevented.
Component composition	The user inserts components from different categories, reorders them in the outline, and modifies their content and style properties.	The screen structure updates immediately, the outline order remains synchronized with the canvas, and the preview reflects the edits without loss of block identity.
Navigation configuration	The user assigns a target screen to an interactive block and then removes or changes that target.	The selected block visibly stores the navigation relation, the outline indicates the presence of a target, and runtime navigation follows the updated destination.
Persistence and compatibility	The user saves a project as JSON and then executes the runner path with both current multi-screen data and an earlier single-screen export.	The current project is serialized with screens, blocks, and navigation targets, while older exports remain loadable through compatibility logic.
SwiftUI export	The user exports a single-screen project and a multi-screen project to SwiftUI source.	The output files are created successfully, and the exported code preserves supported screen structure, styling, and navigation relationships.
Runner execution	The user launches the current prototype through the simulator workflow after saving project state.	The runner application is built, installed, and opened in the iOS Simulator, and the navigable prototype reflects the current saved project.

These criteria strengthen the requirements chapter in two ways. First, they convert broad feature groups into observable end-to-end expectations. Second, they keep the report aligned with the implemented artifact rather than with a purely aspirational future platform. For a visual builder system, that distinction matters: a requirement is only convincing when the resulting editing, export, or runtime behavior can be demonstrated unambiguously.

3 SYSTEM MODELING

This chapter presents the formal models used to describe the behavior, structure, deployment context, and persistence logic of SwiftBuilder. The goal is not to maximize the number of diagrams, but to explain the most important system views in a readable and technically defensible form. The selected models correspond directly to the implemented workflow: multi-screen authoring, contextual editing, navigation wiring, code export, persistent storage, and runner-based execution.

3.1 Modeling Strategy

The modeling set follows UML-oriented conventions and keeps only the views that materially improve understanding of the system [18]. This decision is important for two reasons. First, a strong technical report should avoid large diagrams that add complexity without interpretation. Second, the current codebase already has several distinct subsystems, so the most useful models are those that clarify behavior, structure, and output boundaries rather than attempt to visualize every helper type.

The chapter is organized into three groups:

- behavioral models that describe how the user and the system interact;
- structural models that explain the main entities and their relationships;
- deployment and persistence models that show where the system runs and how project state is transferred.

3.2 Use Case Model

The use case model identifies the main interaction expected from the system. The user is not limited to composing a single screen. Instead, the actor can manage screens, edit components, configure navigation, save project state as JSON, export SwiftUI code, and run the prototype through the runner application. This breadth of interaction is important because it shows that the implemented system supports a full authoring workflow rather than a simple design canvas.

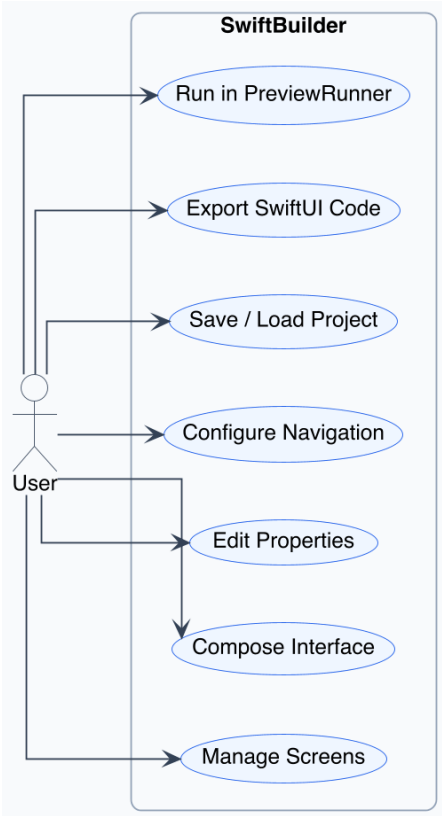

Figure 3.1 - Use Case Diagram of the Main Authoring and Output Workflow

Figure 3.1 aggregates the core user goals of the platform. The system must support visual construction, structural screen management, navigation definition, persistence, code export, and native execution. In requirements terms, the diagram concentrates FR 1 through FR 9 into one readable behavioral summary.

3.3 Activity Model of the Main Workflow

The activity model expands the use case perspective into an operational flow. The process starts with project setup and screen selection, continues through component composition and property editing, and proceeds toward two main output directions: exported SwiftUI code and runtime validation through the runner application. The activity view is useful because it emphasizes that the builder is iterative and that output generation is part of the same workflow rather than a separate external step.

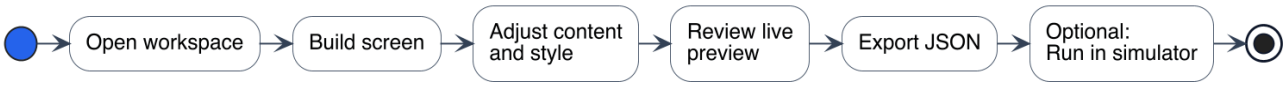

Figure 3.2 - Activity View of the Main Builder Workflow

The activity in Figure 3.2 highlights four key ideas. First, the project is multi-screen from the start, not only block-based. Second, navigation configuration is part of editing rather than a later afterthought. Third, save, export, and runtime validation are integrated into one tool-supported loop. Fourth, the workflow

remains local-first and centered on the builder interface.

3.4 Sequence Model of Runner Execution

The sequence model focuses on the runner-launch path because it is one of the most distinctive technical behaviors of the system. Rather than stopping at visual editing or code export, the platform can serialize the project, mirror it for runtime access, build the runner application, boot the simulator, and launch a navigable prototype.

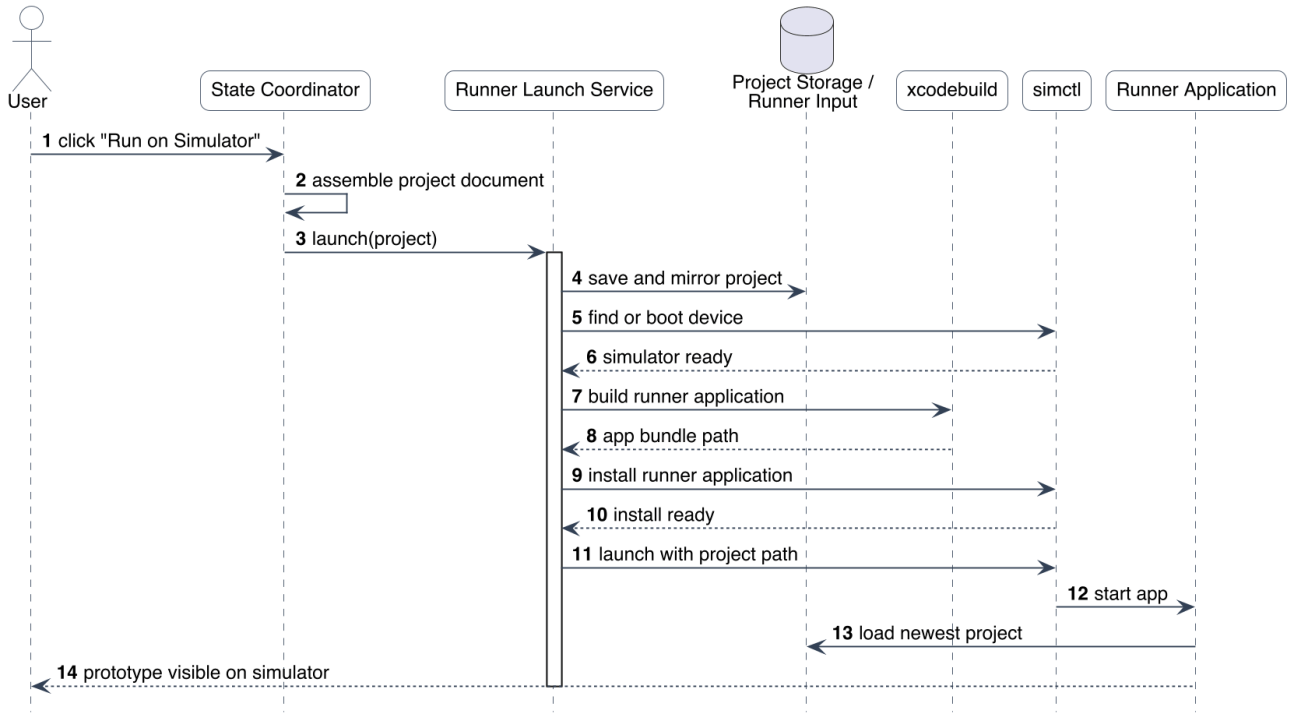

Figure 3.3 - Sequence Diagram of the Runner Launch Flow

Figure 3.3 formalizes the collaboration among the central state coordinator, the runner-launch service, project storage, the Apple build tools, simulator control tools, and the runner application. Its analytical value lies in making the execution boundary explicit. The builder does not merely describe a prototype; it orchestrates its handoff to a runnable iOS environment.

3.5 Class Model of the Core Types

The class model focuses on the minimum set of types required to understand the implemented architecture. The diagram centers on the main state coordinator, the screen and block structures that define the editable model, the exported project representation, and the two output-oriented subsystems: code generation and simulator launch.

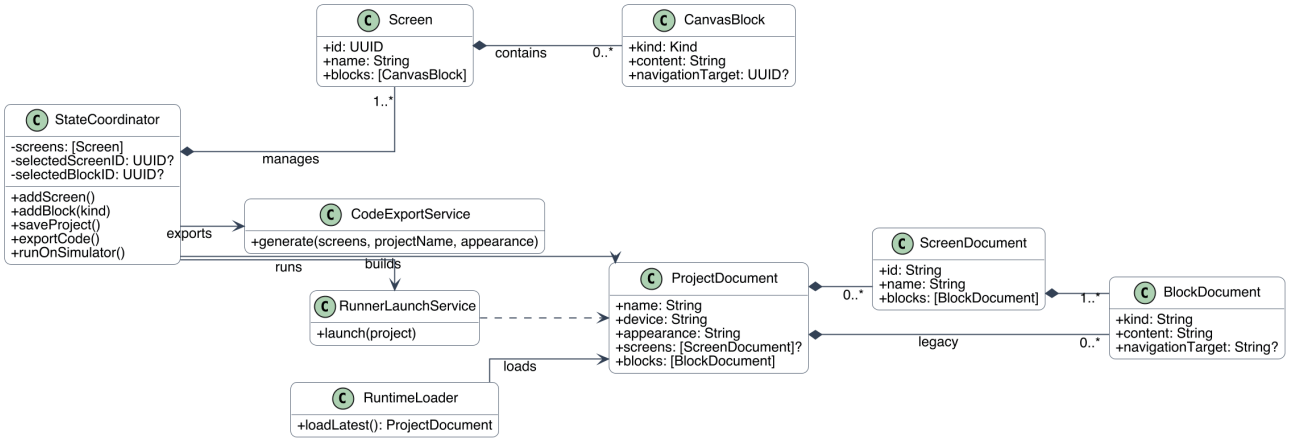

Figure 3.4 - Class Diagram of the Core Builder, Export, and Runner Types

The class view in Figure 3.4 should be read as a conceptual model rather than as an exhaustive reverse engineering artifact. It shows where editable state is owned, how screens and blocks are structured, how project serialization is represented, and which classes are responsible for transformation and runtime launch. This level of abstraction is appropriate for the report because it supports architectural reasoning without overwhelming the reader with minor helper structures.

3.6 Deployment Model

The deployment model explains where the system runs physically and which nodes participate in the builder-runner workflow. The project remains local-first, so the deployment view emphasizes the macOS workstation, project storage, the Xcode toolchain, and the iOS simulator environment.

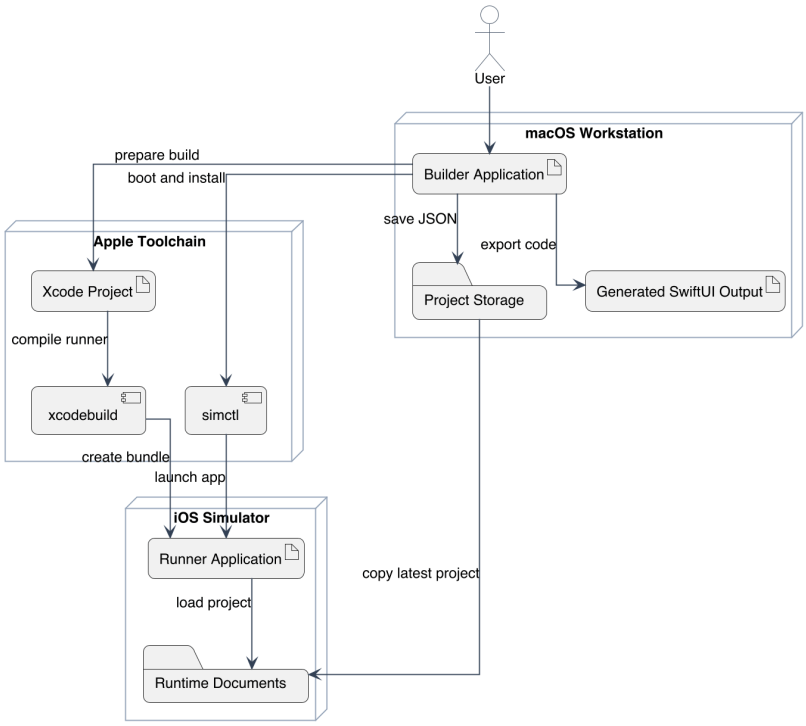

Figure 3.5 - Deployment Diagram of the Local Builder and Runner Environment

The deployment diagram is important because it shows that the architecture does not depend on hidden cloud services or external execution nodes. The builder application, local project data, generated SwiftUI output, build tools, and runner installation all remain within the local Apple-platform workflow. This is consistent with both the thesis scope and the actual implementation.

3.7 Persistence Model

The persistence model documents the JSON structure that transfers project state between authoring and runtime. A classical entity-relationship diagram would be misleading here because the system does not use a relational database. The correct representation is the persistent project document itself.

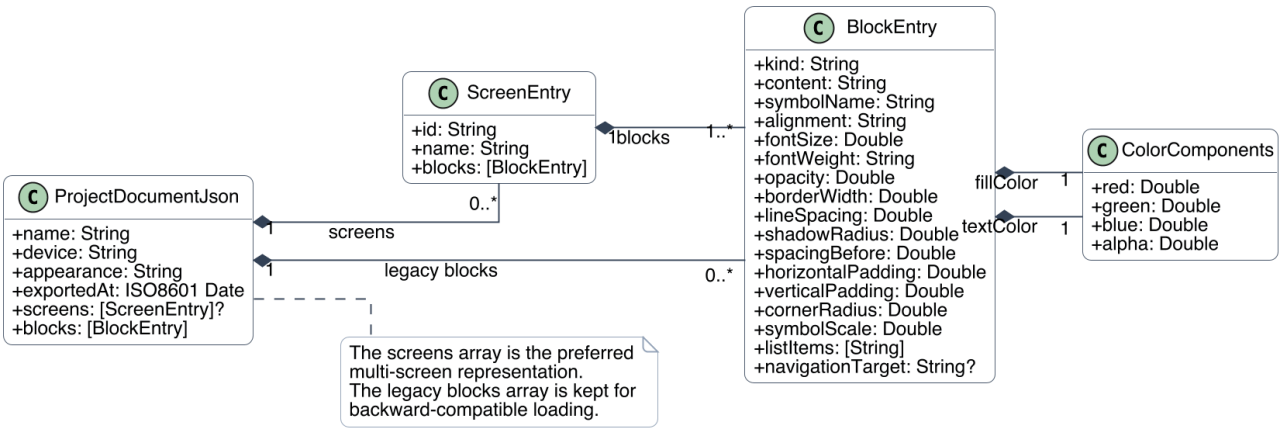

Figure 3.6 - Persistence Model of the Builder Project JSON

Figure 3.6 shows that the root object stores project-level metadata together with both the preferred multi-screen representation and the legacy single-screen block representation. The presence of both structures is important because it documents how the system preserves backward compatibility while evolving from earlier single-screen exports toward the current multi-screen architecture. Navigation targets and component styling are also persisted explicitly, which makes the JSON file a true interchange contract rather than a shallow save file.

3.8 High-Level Architecture Model

At the highest level, the platform is organized into a macOS builder layer, a shared model and rendering layer, a local project representation, an Apple toolchain layer, and an iOS runner layer. The builder itself is further divided into focused modules such as command and navigation controls, a screen-management panel, a component library, a preview canvas, a property inspector, a template selector, a code-export service, and a runner-launch service.

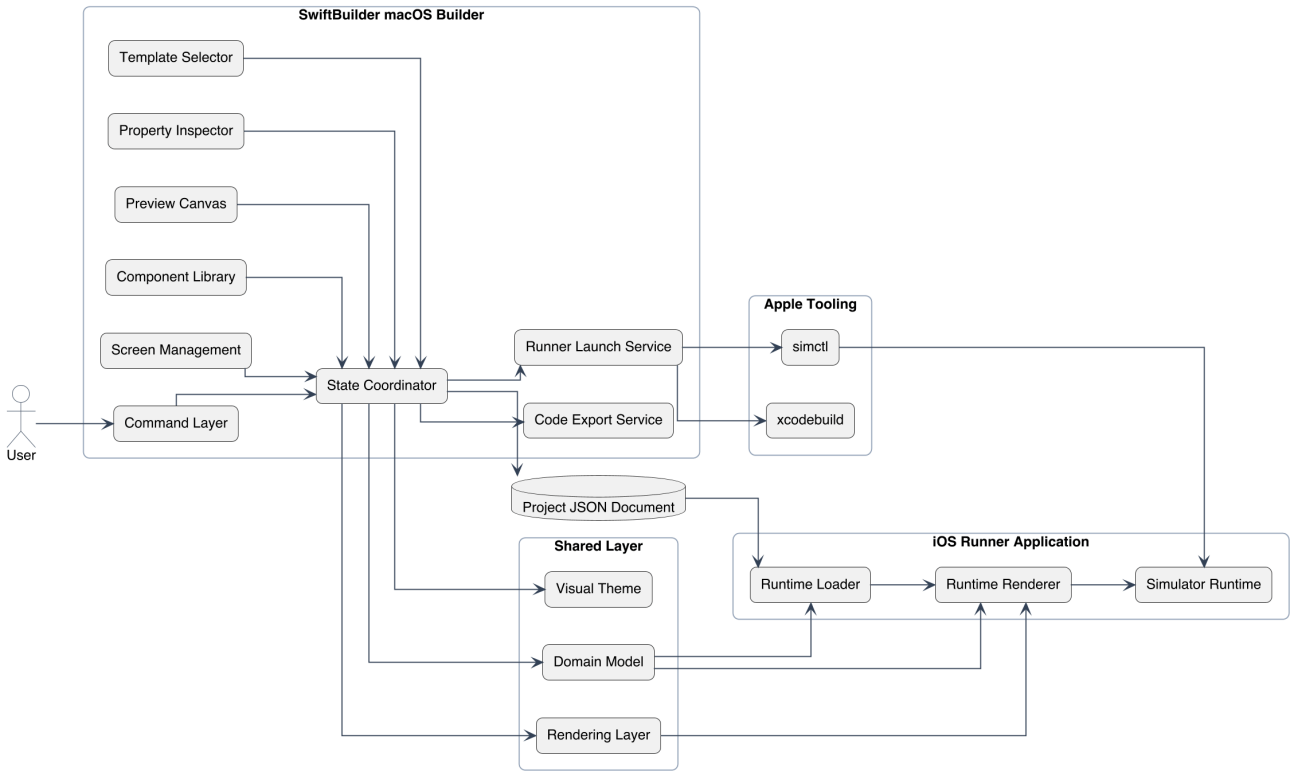

Figure 3.7 - High-Level Architecture of SwiftBuilder

The architecture in Figure 3.7 clarifies why the project should not be interpreted as a monolithic editor. The user-interface modules communicate through a central coordination layer, the shared layer provides cross-target consistency, the persistence model defines the exchange contract, and the export and launch services implement two different output directions. This separation is one of the main reasons why the current implementation already provides a stable foundation for later extension.

3.9 Consistency Between Models and Requirements

The value of the modeling chapter lies not only in the presence of several UML-style views, but also in their internal consistency. If the diagrams described different systems, their academic value would be weak. In the present report, each model addresses a different analytical question while still referring to the same implemented workflow and the same requirement groups.

This consistency can be observed first in the treatment of screen management and interface composition. The use case diagram states that the actor must manage screens and compose interfaces. The activity diagram then translates these goals into an iterative operational flow in which screens are selected, components are added or reordered, properties are edited, and navigation may be assigned. The class model supports the same interpretation by showing that screens and blocks are not incidental user-interface artifacts, but core elements of the persistent project structure. This correspondence is important because it demonstrates that the project is modeled as a multi-screen authoring system end to end, not merely described that way in prose.

Second, the models are consistent in their treatment of output generation. The use case diagram identifies two outputs, namely SwiftUI code export and runtime validation. The sequence diagram expands the runtime path into concrete collaborating elements, while the deployment diagram shows that this path remains fully local and bounded by the Apple toolchain. The high-level architecture model complements these views by explaining why export and runner execution are separate services rather than secondary details hidden inside the interface. Together, these views support FR 7, FR 8, and FR 9 from different perspectives: behavioral intent, execution order, and subsystem separation.

Third, the persistence model connects the behavioral and structural views with the actual interchange contract of the system. A project that can be edited visually but cannot be represented consistently in saved form would not satisfy the thesis objective. The persistence model shows how project metadata, screens, blocks, appearance-relevant properties, and navigation targets are transferred. The class model explains where this structure originates logically, and the deployment model explains how the saved document participates in the builder-to-runner handoff. This is why the persistence diagram is not an accessory to the chapter. It is a central bridge between editing-time structure and runtime reconstruction.

The modeling set also reflects deliberate simplification choices. The report does not include every possible state or collaboration diagram from the earlier exploratory materials, because many of them would duplicate information without improving interpretability. For a bachelor thesis report, readability matters as much as completeness. The chosen diagrams are therefore sufficient not because they are numerous, but because together they answer the essential questions: who interacts with the system, how the main workflow proceeds, which structures carry the editable model, where the system is deployed, and how project state is

preserved across outputs.

3.10 Modeling Synthesis

Taken together, the diagrams in this chapter provide a coherent model set for the implemented system. The use case model defines the main user goals, the activity model explains the authoring flow, the sequence model formalizes runner execution, the class model identifies the core architectural entities, the deployment model bounds the physical environment, the persistence model documents the transfer contract, and the high-level architecture model connects all major subsystems into one readable view.

These diagrams are therefore not included merely to satisfy a formal requirement. They demonstrate that the project has a well-defined internal logic and that the current implementation can be discussed, critiqued, and extended on the basis of explicit models rather than informal descriptions alone.

4 SYSTEM IMPLEMENTATION

This chapter is limited to the technologies used for implementing SwiftBuilder and to the architectural choices needed to justify those technologies. The emphasis is placed on why the selected stack is appropriate for a native *macOS*-to-*iOS* prototyping platform and how it supports the implemented workflow.

4.1 Implementation Context

The implemented solution is developed as a local-first Apple-platform application. It is intended to support visual interface authoring on *macOS* and executable validation on *iOS* Simulator without introducing a backend server, a relational database, or external runtime dependencies.

At repository level, the archived Xcode project still retains a legacy internal identifier in target names, project files, and some build scripts. In the present report, the user-facing prototype is referred to as SwiftBuilder, because this name better reflects the functional role of the system. This distinction is important for technical honesty: the report terminology and some repository internals are not yet fully harmonized, but they refer to the same implemented artifact.

4.2 Technology Stack Overview

Table 4.1 - Technology Stack and Justification

Layer	Technology	Justification
Builder application	Swift + SwiftUI	Enables a native *macOS* interface and keeps the authoring environment close to the target *iOS* presentation technology.
Platform integration	AppKit + Foundation	Supports file dialogs, local file access, process execution, and *macOS*-specific interaction requirements that are not covered by SwiftUI alone.
State coordination	Swift observation model + native undo services	Supports synchronized editing state, undo and redo, and coordinated actions across the screen list, component palette, preview area, and inspector.
Persistence	JSON + Codable	Provides a lightweight, inspectable, and backward-compatible project representation suitable for local storage and runner input [19].
Code generation	Custom SwiftUI export logic	Converts the visual project model into standalone SwiftUI source without depending on third-party code-generation tools.
Runtime validation	iOS runner application + SwiftUI	Enables native *iOS* execution using the same shared model and rendering logic as the builder.
Automation	Xcode command-line tools + simulator control tools	Reuses the standard Apple toolchain for build, installation, and launch automation [7], [20].

4.3 Justification of the Native Apple Stack

Swift and SwiftUI were selected because the thesis subject is explicitly centered on Apple platforms. A web stack or a cross-platform framework would have reduced the closeness between the authoring environment and the target runtime. By using SwiftUI in both the builder and the runner, the system remains aligned with the declarative interface model of modern *iOS* development.

AppKit and Foundation complement SwiftUI where platform services are required. The builder must interact with the local file system, present native save dialogs, coordinate background processes, and invoke Apple development tools. These responsibilities are not incidental; they are necessary for implementing a local-first prototyping workflow that exports code and launches the runner application in the simulator.

JSON was chosen as the project persistence format because it is transparent, portable, and easy to inspect during development. In the current system, the saved document is not only a user-facing storage format, but also the interchange contract between the builder and the runner. The use of `Codable` keeps serialization logic concise and consistent with the native Swift type system.

4.4 Implementation Architecture

The implemented solution follows a dual-target architecture. One target contains the *macOS* builder, where screens are created, organized, and configured. The second target contains the *iOS* runner application, which reads the exported project and renders it as a navigable prototype. This split is appropriate because authoring and execution have different user-interface responsibilities, yet both must remain consistent with the same domain model.

Consistency between the two targets is preserved through a shared model and rendering layer. The builder and the runner rely on the same screen representation, component structures, and visual rendering rules. This reuse is technically important because it reduces divergence between editing-time preview and runtime execution.

The implementation also uses a centralized coordination layer for builder interactions. Screen selection, block selection, preview settings, export actions, and undo history are managed through one observable state model. This architectural decision reduces synchronization problems between the visible interface panels and keeps editing behavior coherent.

The final architectural element is the output pair formed by SwiftUI export and runner execution. The code-generation path supports structured handoff toward later implementation, while the runner path supports immediate validation through Apple tooling. Together, these outputs justify the local-first builder-runner architecture of the project.

4.5 Technical Constraints of the Current Implementation

The chosen technologies also define the current implementation limits. The workflow is specific to *macOS*, SwiftUI, Xcode, and the *iOS* simulator ecosystem. The persistence format is file-based rather than database-based. The generated SwiftUI code is focused on interface structure and navigation, not on backend logic or full application semantics. In addition, the project remains intentionally free of external runtime dependencies, which improves reproducibility but limits the adoption of third-party interface or synchronization frameworks.

These constraints are acceptable for the thesis subject because the project is not intended to be a universal no-code platform. It is intended to be a native visual environment for early-stage *iOS* interface prototyping, structured export, and simulator-backed validation.

4.6 Technology Selection Criteria

The selected technology stack was not chosen arbitrarily. It was evaluated against several criteria derived from the thesis subject and from the target use case of the implemented system.

The first criterion was native fidelity. Because the platform is intended for interface prototyping in the Apple ecosystem, the builder had to remain close to the semantics, layout conventions, and rendering behavior of *iOS* itself. This requirement strongly favored SwiftUI over web-based or cross-platform authoring stacks. A browser-based builder could have offered broader portability, but it would also have introduced an additional translation layer between the authoring environment and the target runtime. For the current thesis, that trade-off would have weakened the most important technical promise of the project.

The second criterion was local autonomy. The system is expected to operate without cloud infrastructure, mandatory accounts, or remote execution services. This requirement influenced the selection of JSON persistence, native file-system access, and Apple command-line tooling. It also explains why the implementation does not rely on hosted preview services or external orchestration frameworks. A local-first stack is not only simpler operationally; it is also more transparent for academic evaluation because the entire workflow can be reproduced on one workstation.

The third criterion was architectural coherence. The same conceptual model of screens, components, and navigation targets had to remain valid across editing, preview, export, and runtime reconstruction. Technologies that would fragment the model between unrelated representations were therefore less suitable. Swift types with `Codable` support, shared rendering logic, and Apple-native build tools provide a more coherent solution because they allow one project representation to flow through multiple stages of the pipeline without semantic reinvention.

The fourth criterion was educational and maintenance value. A bachelor thesis project should not

only work; it should also be understandable, extendable, and defensible. The native Apple stack satisfies this requirement because its components are well documented, widely recognizable, and closely related to professional iOS development practice. This makes the resulting architecture easier to analyze academically and easier to extend in later iterations.

4.7 Role of the Stack in the End-to-End Workflow

The practical adequacy of the selected technologies becomes clearer when they are viewed as one continuous workflow rather than as isolated tools. The builder interface on *macOS* is responsible for authoring tasks such as screen management, component composition, property editing, and navigation assignment. SwiftUI supports the visible editing environment, while AppKit and Foundation provide the system-level services required for file access, document export, and process control.

Once the authoring state is established, the persistence layer translates that state into a JSON project document. At this moment, JSON and `Codable` act as a bridge rather than merely as a storage format. They preserve the structure of the editable model in a way that can be reused by both the export subsystem and the runner application. This means that persistence, export, and runtime validation all depend on the same project representation, which is one of the key reasons why the chosen stack remains coherent.

The export path then transforms the same project model into SwiftUI source code. This stage justifies the presence of custom export logic inside the implementation stack. Generic code-generation tooling was not necessary because the target language, the data model, and the output scope are all narrowly defined by the thesis subject. A dedicated exporter is therefore both simpler and more transparent than a broader transformation framework.

The runtime-validation path uses the Apple toolchain to build and launch the runner application in the *iOS* Simulator. In this stage, the Xcode command-line tools and simulator-control utilities are not secondary operational details; they are technologies that directly enable the thesis contribution of executable prototype validation. Without them, the project would stop at visual authoring and code export. With them, the system closes the loop between design-time editing and native runtime behavior.

When considered together, these phases show why the stack must be justified as a complete implementation chain. Swift and SwiftUI define the native authoring and runtime surfaces, Foundation and AppKit connect the builder to the local operating environment, JSON and `Codable` preserve the project contract, and the Apple build tools make executable validation possible. This combination is therefore not merely convenient. It is structurally aligned with the thesis objective.

CONCLUSIONS

The report documents SwiftBuilder as a substantial macOS-native platform for visual prototyping of iOS applications. The implemented system supports multi-screen authoring, navigation wiring, screen templates, device-aware preview, shared rendering, SwiftUI code export, and runner-based execution inside one coherent Apple-platform workflow.

The main achievements described in the report can be summarized as follows:

- the domain gap between static design tools, generic no-code environments, and native iOS development was analyzed and positioned more precisely;
- the requirements specification was aligned with the real feature set of the builder, including screen management, navigation, export, persistence, and automation;
- the modeling chapter describes the implemented architecture through behavioral, structural, deployment, and persistence views;
- the implementation chapter explains the native Apple-platform technology stack, the dual-target architecture, the local-first persistence model, and the automation path used for simulator validation.

The current implementation is already strong enough to support the thesis direction, but several limitations remain:

- the generated output focuses on interface structure and navigation rather than on business logic, backend integration, or full application semantics;
- the component catalog is curated rather than fully open-ended, which improves consistency but limits arbitrary custom UI construction;
- the workflow remains specific to macOS, SwiftUI, Xcode, and the Apple simulator ecosystem;
- collaboration, cloud synchronization, and shared project workflows are outside the present system boundary.

These limitations do not weaken the value of the project; instead, they clarify its real technical niche. The system is strongest when evaluated as a native visual builder and structured handoff environment for early iOS interface design, not as a universal no-code platform.

The present implementation also defines a clear continuation path. Because the builder, persistence contract, export logic, and runner execution path already form a coherent chain, later work can focus on deepening capability rather than redefining architecture. The most natural directions include richer component semantics, stronger navigation patterns, broader preview controls, and more expressive export output. In addition, the local-first project representation creates a stable basis for future extensions such as design tokens, reusable style presets, or stronger validation of unsupported combinations before export and runtime

execution.

From an engineering perspective, the project is especially valuable because it demonstrates that the distance between visual composition and native execution can be reduced without relying on a cloud platform or a generic cross-platform abstraction. This is a meaningful result for the thesis topic. It shows that a macOS-native authoring tool can remain close to the real target environment of iOS applications while still giving the user a faster and more structured workflow than manual SwiftUI construction from the first step. The project therefore contributes not only an implemented prototype, but also a concrete architectural argument for why shared rendering, JSON-based interchange, export support, and simulator-backed validation belong together in one system.

In conclusion, the thesis is supported by a coherent architectural and implementation base and positions SwiftBuilder as a credible academic and practical contribution at the intersection of low-code design tools, native Apple-platform development, and executable interface prototyping.

BIBLIOGRAPHY

[1] A. C. Bock and U. Frank, "Low-code platform", *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021. DOI: 10.1007/s12599-021-00726-8

[2] D. D. Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?", *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022. DOI: 10.1007/s10270-021-00970-2

[3] A. Viljoen, A. Hein, and H. Krcmar, "Low-code development platform ecosystems", *Electronic Markets*, vol. 36, p. 12, 2026. DOI: 10.1007/s12525-025-00848-x

[4] B. Binzer and T. J. Winkler, "Democratizing software development: A systematic multivocal literature review and research agenda on citizen development", in *Proceedings of the 13th International Conference on Software Business*, ser. Lecture Notes in Business Information Processing, vol. 463, 2022, pp. 244–259. DOI: 10.1007/978-3-031-20706-8_17

[5] Apple Inc. "The swift programming language", Accessed: Nov. 12, 2025. [Online]. Available: https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[6] Apple Inc. "Swiftui documentation", Accessed: Nov. 21, 2025. [Online]. Available: https://developer.apple.com/documentation/swiftui

[7] Apple Inc. "Xcode documentation", Accessed: Dec. 3, 2025. [Online]. Available: https://developer.apple.com/documentation/xcode

[8] Apple Inc. "Human interface guidelines", Accessed: Dec. 14, 2025. [Online]. Available: https://developer.apple.com/design/human-interface-guidelines

[9] S. Ogata and S. Matsuura, "A review method for UML requirements analysis model employing system-side prototyping", *SpringerPlus*, vol. 2, p. 134, 2013. DOI: 10.1186/2193-1801-2-134

[10] S. D. Nelson, G. D. Fiol, H. Hanseler, B. I. Crouch, and M. R. Cummins, "Software prototyping: A case report of refining user requirements for a health information exchange dashboard", *Applied Clinical Informatics*, vol. 7, no. 1, pp. 22–32, 2016. DOI: 10.4338/ACI-2015-07-CR-0091

[11] A. Bucaioni, A. Cicchetti, and F. Ciccozzi, "Modelling in low-code development: A multi-vocal systematic review", *Software and Systems Modeling*, vol. 21, pp. 1959–1981, 2022. DOI: 10.1007/s10270-021-00964-0

[12] K.-K. Kemell, A. Nguyen-Duc, M. Suoranta, and P. Abrahamsson, "Startcards — a method for early-stage software startups", *Information and Software Technology*, vol. 160, p. 107 224, 2023. DOI: 10.1016/j.infsof.2023.107224

[13] Figma, Inc. "Figma", Accessed: Jan. 11, 2026. [Online]. Available: https://www.figma.com/

[14] Sketch B.V. "Sketch", Accessed: Jan. 19, 2026. [Online]. Available: https://www.sketch.com/

[15] FlutterFlow, Inc. "Flutterflow", Accessed: Jan. 27, 2026. [Online]. Available: https://flutterflow.io/

[16] Adalo. "Adalo", Accessed: Feb. 2, 2026. [Online]. Available: https://www.adalo.com/

[17] Judo. "Meet judo: The design and build tool for swiftui apps", Accessed: Feb. 14, 2026. [Online]. Available: https://www.judo.app/the-mac-app/

[18] Object Management Group, *Unified modeling language specification, version 2.5.1*, 2021. Accessed: Mar. 1, 2026. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[19] ECMA International. "Ecma-404: The json data interchange syntax", Accessed: Feb. 25, 2026. [Online]. Available: https://ecma-international.org/publications-and-standards/standards/ecma-404/

[20] Apple Inc. "Running your app in simulator or on a device", Accessed: Dec. 22, 2025. [Online]. Available: https://developer.apple.com/documentation/xcode/running-your-app-in-simulator-or-on-a-device